

PulseDesk

Market Data Service

Documentação Técnica

Arquitetura baseada em Microserviços e Eventos



Sumário

1	Visão Geral	2
1.1	Responsabilidade	2
1.2	Papel na arquitetura	2
1.3	Principais responsabilidades	2
2	Estrutura do Microserviço	3
3	Fluxo Principal	4
4	Análise dos Arquivos	4
4.1	MarketDataServiceApplication.java	4
4.2	MarketDataController.java	4
4.3	MarketDataService.java	5
4.4	DTO	5
4.5	ExternalMarketDataProvider.java	6
4.6	FinnhubMarketDataProvider.java	6
4.7	FinnhubQuoteResponse.java	6
4.8	AlphaVantageHistoricalMarketDataProvider.java	7
4.9	MarketDataEventProducer.java	7
4.10	HistoricalMarketDataEventProducer.java	8
5	Integrações Externas	8
5.1	Finnhub	8
5.2	Alpha Vantage	9
6	Comunicação por Eventos	9
6.1	Eventos produzidos	9
6.2	Motivação	10
7	Configuração	10
7.1	application.yml	10
8	Dependências	11
8.1	pom.xml	11
9	Docker	11

1 Visão Geral

1.1 Responsabilidade

O Market Data Service é o microserviço responsável por centralizar a obtenção e a distribuição de dados de mercado utilizados pelo PulseDesk.

Ele consulta fontes externas para obter tanto cotações atuais quanto preços históricos de ativos. Depois que essas informações são recebidas, o serviço as transforma para os formatos internos utilizados pelo sistema, disponibiliza parte dos dados através de endpoints HTTP e publica atualizações no Kafka.

Dessa forma, os demais componentes do PulseDesk não precisam conhecer diretamente os detalhes das APIs Finnhub e Alpha Vantage. A comunicação com esses provedores fica concentrada no Market Data Service.

1.2 Papel na arquitetura

Dentro da arquitetura do PulseDesk, o Market Data Service funciona como a fronteira entre as fontes externas de market data e os componentes internos da aplicação.

Para consultas síncronas, o frontend envia uma requisição HTTP ao MarketData-Controller. O Controller encaminha a solicitação ao MarketDataService, que utiliza os providers responsáveis por acessar as APIs externas e devolve a resposta ao frontend.

Além disso, o serviço também participa da arquitetura orientada a eventos. Depois de obter informações de mercado, os producers transformam os dados em eventos Avro e os publicam no Kafka. Outros microserviços podem consumir esses eventos sem que o Market Data Service precise chamá-los diretamente.

Assim, o serviço possui dois tipos principais de saída:

- respostas HTTP, utilizadas quando existe uma requisição direta do frontend;
- eventos Kafka, utilizados para propagar atualizações para outros componentes do sistema.

1.3 Principais responsabilidades

- Obter cotações atuais de ativos através do Finnhub;
- Obter histórico diário de preços através do Alpha Vantage;
- Normalizar e validar os símbolos recebidos;
- Converter respostas externas para os DTOs internos do PulseDesk;
- Disponibilizar dados através de API REST;
- Publicar atualizações atuais e históricas no Kafka;

- Evitar consultas históricas repetidas através de cache;
- Atualizar periodicamente ativos monitorados quando o scheduling está ativo.

2 Estrutura do Microserviço

O Market Data Service é organizado em pacotes que separam as diferentes responsabilidades da aplicação.

```
market-data-service/  
|  
|-- controller/  
|   '-- MarketDataController.java  
|  
|-- dto/  
|   |-- HistoricalPriceResponse.java  
|   '-- MarketDataResponse.java  
|  
|-- producer/  
|   |-- HistoricalMarketDataEventProducer.java  
|   '-- MarketDataEventProducer.java  
|  
|-- provider/  
|   |-- ExternalMarketDataProvider.java  
|   |-- FinnhubMarketDataProvider.java  
|   |-- FinnhubQuoteResponse.java  
|   '-- AlphaVantageHistoricalMarketDataProvider.java  
|  
|-- service/  
|   '-- MarketDataService.java  
|  
'-- MarketDataServiceApplication.java
```

O pacote controller concentra a entrada HTTP do microserviço.

O pacote service contém a lógica de orquestração, decidindo quais providers devem ser utilizados e quando eventos precisam ser publicados.

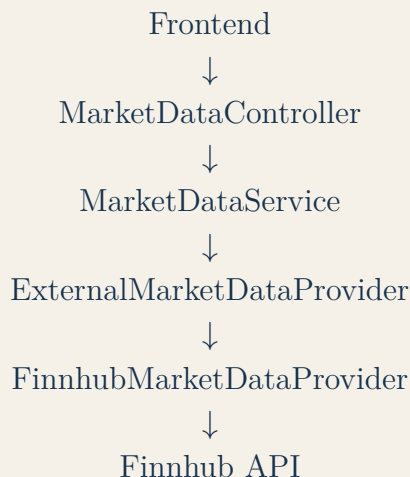
O pacote provider concentra as integrações com fontes externas de market data. Dessa forma, detalhes específicos do Finnhub e do Alpha Vantage permanecem separados da lógica principal da aplicação.

O pacote dto contém as estruturas utilizadas para transportar os dados dentro do PulseDesk e para as respostas HTTP.

Por fim, o pacote producer contém os componentes responsáveis por transformar os dados em eventos e publicá-los nos tópicos Kafka.

3 Fluxo Principal

O fluxo principal de uma consulta de cotação começa quando o frontend solicita informações de determinado símbolo ao Market Data Service.



O FinnhubMarketDataProvider recebe a resposta externa, converte os campos retornados pelo Finnhub para a representação utilizada pelo PulseDesk e devolve um MarketDataResponse ao MarketDataService.

O serviço pode então utilizar esse resultado de duas maneiras. A primeira é devolvê-lo ao Controller para que a informação seja retornada ao frontend através da resposta HTTP. A segunda é encaminhar os dados ao MarketDataEventProducer, responsável por criar um evento MarketDataUpdated e publicá-lo no Kafka.

4 Análise dos Arquivos

4.1 MarketDataServiceApplication.java

MarketDataServiceApplication é a classe de entrada do microserviço. Sem esse arquivo, não seria possível inicializar normalmente o Spring Boot dessa aplicação.

O @SpringBootApplication inicializa a configuração do Spring Boot, incluindo component scanning e autoconfiguração.

O main chama SpringApplication.run, que cria o contexto Spring e inicializa os beans (objetos gerenciados pelo Spring) da aplicação.

4.2 MarketDataController.java

MarketDataController é a porta de entrada HTTP do Market Data Service. Ele recebe as requisições de informações de mercado (como o preço de um ativo, histórico, variação percentual), repassa essa necessidade para o MarketDataService e devolve a resposta.

Ele fica separado do `MarketDataProvider` para melhorar a organização do projeto. Caso contrário, receber HTTP e decidir como buscar market data estariam juntos e misturando responsabilidades.

O `@CrossOrigin`, também conhecido como CORS, permite que o frontend (porta 4200) faça requisições ao backend do Market Data Service (porta 8081). Isso ocorre uma vez que, para o navegador, requisições devem possuir uma mesma origem/ porta. Dessa forma, é necessário definir manualmente quando essa regra deve ser violada.

Já o `@RestController` indica que os valores retornados pelo método devem ser transformados em corpo da resposta HTTP. Nesse caso, por exemplo, o Spring Boost serializa o objeto devolvido para JSON.

Além disso, temos o `@RequestMapping`, que indica um prefixo comum para todos os endpoints da classe. Depois, dentro da função, com o `@GetMapping`, é definido o resto do endereço de onde a informação será extraída. Por fim, o `@PathVariable` informa ao Spring para pegar o valor presente naquele segmento do URL e colocar em uma determinada variável.

4.3 `MarketDataService.java`

O `MarketDataService` é camada responsável pela orquestração dos dados de mercado. O serviço normaliza os símbolos consultados, mantém um conjunto thread-safe de ativos monitorados, consulta providers de dados atuais e históricos e utiliza produtores Kafka para publicar atualizações. Um método agendado atualiza periodicamente as cotações dos símbolos previamente consultados.

Nessa classe existem quatro dependências principais: `ExternalMarketDataProvider` obtém a cotação atual; `HistoricalMarketDataProvider` obtém o histórico das cotações; já `MarketDataEvent Producer` e `HistoricalMarketDataEventProducer` produzem eventos no Kafka.

O método `getQuote` adquire o histórico de preços de fechamento do símbolo nos últimos 20 dias e adquire as informações mais atualizadas do símbolo e as publica como um evento no Kafka.

Já o método `normalizeSymbol` é importante para verificar se aquele ativo existe, seguindo um padrão dos nomes dos ativos e não aceitando valores nulos.

O método `fetchAndPublish` busca as informações atuais do símbolo e publica o evento no Kafka.

Por fim, existem dois métodos que checam informações históricas do ativo. O `publishHistoricalData` adquire as informações dos últimos 21 dias para gerar o dado sobre a volatilidade de um preço. Já o `getHistoricalPrices` devolve os preços de fechamento dos últimos 100 dias de um ativo, informação utilizada na construção do gráfico.

4.4 DTO

A utilização de DTOs permite separar a representação externa dos dados das implementações internas do serviço. Dessa forma, o formato utilizado na comunicação com

outros componentes do sistema não precisa estar diretamente acoplado às classes responsáveis pela lógica de negócio ou pela integração com provedores externos.

No serviço de dados de mercado, os principais DTOs expostos pela aplicação são `MarketDataResponse` e `HistoricalPriceResponse`.

4.5 `ExternalMarketDataProvider.java`

A interface `ExternalMarketDataProvider` define um contrato para qualquer provedor externo de cotação em tempo real. Ela está no pacote `provider` uma vez que ela obtém dados de fontes externas.

Vale ressaltar que, em Java, o papel de uma interface é definir um conjunto de métodos que uma classe deve obrigatoriamente executar, especificando o que deve ser feito sem ditar como.

Logo, para ser um `External Market Data Provider`, a classe precisa ter um método `fetchQuote`.

4.6 `FinnhubMarketDataProvider.java`

Classe responsável por buscar no `Finnhub` uma cotação e transformar a resposta externa em um formato compatível com o `PulseDesk`. Define como essa busca de dados será feita.

O `restClient` é o cliente HTTP utilizado pela classe para enviar requisições à API da `Finnhub` e receber suas respostas. O `FinnhubMarketDataProvider` usa o `restClient` do `Spring Framework` para fazer as chamadas HTTP para o `Finnhub`.

O método `RestClient.builder()` inicia a construção e a configuração de um cliente HTTP. Nesse processo, o objeto `Builder` permite definir propriedades do cliente antes de sua criação efetiva; no caso do `FinnhubMarketDataProvider`, o método `baseUrl()` estabelece a URL base da API da `Finnhub`. Em seguida, o método `build()` finaliza essa configuração e cria a instância de `RestClient` que será utilizada para realizar as requisições HTTP.

4.7 `FinnhubQuoteResponse.java`

`FinnhubQuoteResponse` representa exatamente a forma como os dados de cotação são recebidos da API do `Finnhub`.

Essa classe funciona como um DTO externo, ou seja, ela não representa ainda o formato utilizado internamente pelo `PulseDesk`. Sua função é apenas receber e organizar os dados enviados pelo `Finnhub` antes que eles sejam convertidos para `MarketDataResponse`.

O `@JsonIgnoreProperties(ignoreUnknown = true)` informa ao Jackson que campos presentes no JSON do `Finnhub`, mas que não foram declarados nessa classe, podem ser ignorados. Isso é importante porque a resposta do `Finnhub` possui mais informações do que aquelas utilizadas atualmente pelo `PulseDesk`. Já o `@JsonProperty` é utilizado para

fazer a correspondência entre os nomes dos campos presentes no JSON e os nomes das variáveis Java.

A separação entre `FinnhubQuoteResponse` e `MarketDataResponse` é importante porque evita que o restante do PulseDesk fique dependente do formato específico utilizado pelo Finnhub.

4.8 AlphaVantageHistoricalMarketDataProvider.java

Classe responsável por buscar os dados históricos dos ativos no Alpha Vantage e transformá-los em um formato compatível com o PulseDesk.

Diferentemente do `FinnhubMarketDataProvider`, que busca a cotação mais recente do ativo, o `HistoricalMarketDataProvider` trabalha com uma sequência de preços históricos de fechamento.

A constante `REQUIREDPRICES` define a quantidade mínima de preços históricos necessários para alguns cálculos do sistema (os dos dados analíticos de volatilidade).

O `restClient` é o cliente HTTP utilizado pela classe para enviar requisições à API da Alpha Vantage e receber suas respostas.

A variável `cache` armazena temporariamente os dados históricos que já foram buscados. Ela utiliza um `ConcurrentHashMap`, permitindo que diferentes execuções do serviço acessem esse mapa de maneira segura mesmo em um ambiente com múltiplas threads. O `cache` utiliza o símbolo do ativo como chave e um objeto `CachedHistory` como valor. Esse objeto armazena a data em que os dados foram buscados e a lista de preços históricos recebida.

O método `getRecentDailyCloses()` é utilizado quando o sistema precisa apenas dos 21 preços de fechamento mais recentes.

O método `getDailyCloseHistory()` é responsável por obter o histórico diário completo do ativo.

O método `fetchFromAlphaVantage()` é responsável por realizar efetivamente a requisição HTTP ao Alpha Vantage.

4.9 MarketDataEventProducer.java

O `MarketDataEventProducer` é responsável por publicar no Kafka as atualizações de cotação obtidas pelo Market Data Service.

O `KafkaTemplate` é fornecido pelo Spring Kafka e nele é definido o tipo da chave da mensagem (`String`) e o tipo de valor/evento (`MarketDataUpdated`).

O `newBuilder()` começa a construção do evento, o `set()` preenche os campos e o `build()` cria o evento final. Depois que os campos são preenchidos, a classe Java é gerada a partir de um schema Avro que define a estrutura desse evento.

Um UUID é criado para identificar cada evento e, depois, ele é enviado no tópico `market-data.updated`. O símbolo é usado para que eventos com a mesma chave sejam direcionados a mesma partição do tópico, o que ajuda a preservar a ordem das atualizações daquele ativo dentro da partição.

4.10 HistoricalMarketDataEventProducer.java

Segue o mesmo padrão que o MarketDataEventProducer, mudando apenas o tópico e que a publicação é uma série de preços históricos.

5 Integrações Externas

O Market Data Service utiliza dois provedores externos. O Finnhub é utilizado para obter a cotação mais recente dos ativos, enquanto o Alpha Vantage fornece a série histórica diária de preços de fechamento.

A separação entre os providers permite que detalhes específicos de cada API permaneçam isolados do restante da aplicação.

5.1 Finnhub

O Finnhub é utilizado pelo PulseDesk para consultar a cotação atual de um ativo.

A comunicação é realizada pelo FinnhubMarketDataProvider utilizando o RestClient do Spring. O cliente possui como URL base:

```
https://api.finnhub.io/api/v1
```

Para consultar um ativo, é feita uma requisição GET ao endpoint:

```
/quote
```

O símbolo é enviado como query parameter. Conceitualmente, uma consulta para AAPL possui a seguinte forma:

```
GET /quote?symbol=AAPL
```

A autenticação é realizada através da chave da API enviada no header X-Finnhub-Token.

A resposta do Finnhub é recebida em JSON. Entre os campos utilizados pelo PulseDesk estão:

- c: preço atual;
- dp: variação percentual;
- t: timestamp da cotação.

Esses nomes pertencem ao contrato externo do Finnhub. O Jackson utiliza as anotações presentes em FinnhubQuoteResponse para associar os campos do JSON às variáveis Java.

Depois disso, o FinnhubMarketDataProvider valida a resposta, converte o timestamp para Instant e cria um MarketDataResponse. A partir desse ponto, o restante do PulseDesk não precisa conhecer o formato específico utilizado pelo Finnhub.

5.2 Alpha Vantage

O Alpha Vantage é utilizado para obter o histórico diário de preços de fechamento.

A comunicação é realizada pelo `AlphaVantageHistoricalMarketDataProvider`, também através de `RestClient`.

A requisição é enviada ao endpoint `/query` utilizando a função `TIME_SERIES_DAILY`. Entre os parâmetros enviados estão:

- `function=TIME_SERIES_DAILY`, que solicita a série temporal diária;
- `symbol`, que identifica o ativo;
- `outputsize=compact`, que solicita o conjunto recente de observações;
- `datatype=csv`, que define o formato da resposta;
- `apikey`, utilizada para autenticação.

Diferentemente do Finnhub, nessa integração a resposta utilizada pelo PulseDesk é recebida em CSV.

A primeira linha corresponde ao cabeçalho. As demais linhas representam observações diárias e são convertidas para `HistoricalPriceResponse`, utilizando a data e o preço de fechamento.

Antes de fazer uma nova chamada ao Alpha Vantage, o provider consulta seu cache. Caso o histórico daquele símbolo já tenha sido obtido no mesmo dia, o valor armazenado é reutilizado. Essa estratégia reduz chamadas externas repetidas e evita consumir desnecessariamente o limite da API.

6 Comunicação por Eventos

Além de responder requisições HTTP, o Market Data Service publica eventos no Kafka.

Essa comunicação é assíncrona. Depois que o producer envia um evento ao tópico, o Market Data Service não precisa chamar diretamente cada serviço interessado naquela atualização. Os consumidores inscritos no tópico recebem e processam os eventos de forma independente.

6.1 Eventos produzidos

O Market Data Service produz dois tipos principais de evento:

- `market-data.updated`: publica uma atualização da cotação atual de um ativo;
- `historical-market-data.updated`: publica uma sequência de preços históricos de fechamento.

6.2 Motivação

A publicação de eventos reduz o acoplamento entre os microsserviços.

Sem Kafka, o Market Data Service precisaria conhecer diretamente cada componente interessado nas atualizações de mercado e realizar chamadas específicas para eles.

Com a arquitetura orientada a eventos, o Market Data Service conhece apenas o Kafka e o tópico no qual deve publicar.

Isso permite adicionar ou remover consumidores sem alterar o producer. Por exemplo, um serviço de Analytics e um WebSocket Gateway podem reagir ao mesmo evento de market data de maneiras diferentes.

O Kafka é responsável pelo transporte das mensagens, enquanto o Avro define sua estrutura. O Schema Registry complementa esse fluxo armazenando os schemas e permitindo controlar a compatibilidade entre diferentes versões dos contratos.

7 Configuração

7.1 application.yml

O application.yml concentra configurações externas do Market Data Service. Dessa forma, valores que dependem do ambiente não precisam ficar fixados diretamente nas classes Java.

Entre as configurações mais importantes estão o nome da aplicação, a porta HTTP, o endereço do Kafka e as credenciais utilizadas pelos providers externos.

O Market Data Service utiliza a porta 8081. Essa porta identifica o servidor HTTP do serviço e é utilizada pelo frontend para acessar seus endpoints REST.

A configuração do Kafka informa o endereço do broker ao Spring. Em execução local, o endereço pode apontar para localhost, enquanto dentro da rede Docker o serviço utiliza o nome do container Kafka. Essa diferença é resolvida através de variáveis de ambiente, evitando alterar o código Java entre os ambientes.

As propriedades utilizadas pelos providers incluem:

```
finnhub.api-key  
alpha-vantage.api-key
```

Esses valores são lidos nas classes através de @Value. As chaves reais não devem ser armazenadas diretamente no repositório. O padrão do projeto é obtê-las através das variáveis FINNHUB_API_KEY e ALPHA_VANTAGE_API_KEY.

O arquivo também contém a configuração necessária para a comunicação Kafka e, quando aplicável, para a serialização Avro e o acesso ao Schema Registry.

A periodicidade do scheduler não precisa obrigatoriamente estar no application.yml. Quando o intervalo é definido diretamente na anotação @Scheduled, essa configuração pertence ao código da aplicação.

O `application.yml`, portanto, funciona como a camada de configuração do serviço, enquanto as classes Java utilizam essas propriedades sem precisar conhecer de onde seus valores foram obtidos.

8 Dependências

8.1 `pom.xml`

O `pom.xml` é o arquivo utilizado pelo Maven para descrever o projeto Java e controlar suas dependências, plugins e processo de build.

No Market Data Service, as dependências mais importantes são aquelas relacionadas ao Spring Web, Kafka e Avro.

O Spring Web fornece a infraestrutura necessária para os endpoints REST, incluindo as anotações utilizadas pelo Controller, a serialização de respostas para JSON e o `RestClient` utilizado para consultar as APIs externas.

O Spring for Apache Kafka fornece o `KafkaTemplate` utilizado pelos producers para enviar eventos aos tópicos Kafka.

O Apache Avro e as bibliotecas de serialização associadas permitem que os eventos sejam representados a partir de schemas definidos previamente. Dessa forma, `MarketDataUpdated` e `HistoricalMarketDataUpdated` seguem contratos compartilhados entre produtores e consumidores.

A integração com o Schema Registry permite que esses schemas sejam registrados e versionados, reduzindo o risco de produtores e consumidores utilizarem formatos incompatíveis.

O projeto também utiliza os contratos compartilhados do PulseDesk, onde ficam as classes de eventos geradas a partir dos arquivos Avro.

O Market Data Service não necessita de Spring Data JPA, PostgreSQL Driver ou Flyway, pois sua responsabilidade não envolve persistência de dados em banco durante o fluxo atual. Os dados históricos reutilizados pelo provider são mantidos em cache em memória.

Além das dependências, o `pom.xml` também define informações como versão do Java e plugins necessários para compilar e empacotar a aplicação. Não é necessário analisar cada linha desse arquivo para compreender a responsabilidade arquitetural do serviço.

9 Docker

O `Dockerfile` define como o Market Data Service é transformado em uma imagem Docker executável.

A imagem reúne a aplicação Java e o ambiente necessário para executá-la, permitindo que o microserviço rode de forma isolada dos demais componentes.

Durante o processo de build, o código do Market Data Service é compilado e o artefato executável da aplicação Spring Boot é preparado para execução dentro do container.



Quando o container é iniciado, o processo Java executa o `MarketDataServiceApplication` e o Spring Boot inicializa o contexto da aplicação, lê as configurações e disponibiliza o servidor HTTP.

No compose do PulseDesk, o Market Data Service é construído a partir do diretório `backend/market-data-service` e utiliza seu próprio Dockerfile.

A porta 8081 do serviço é publicada para permitir acesso a partir da máquina host.

Dentro da rede Docker, o serviço não utiliza localhost para acessar o Kafka. Ele se comunica com o broker através do nome do serviço Kafka e da porta interna configurada para essa rede.

O Docker Compose também injeta variáveis de ambiente no container, permitindo configurar endereços e credenciais sem colocá-los diretamente na imagem.

A separação é, portanto:

- Dockerfile: define como construir a imagem de um único serviço;
- `compose.yaml`: define como esse container é executado junto com Kafka, Schema Registry, frontend e os demais microsserviços.